GUARDIAN MISSION

Full System Digital Twin

Prepared by:

Mohamed Nabil Ali

Prepared for:

Eng. Ahmed Hassan

www.GitHubRepo

Table of Contents

Project Overview & Objective 3
System Architecture & Methodology 3
Scope of Implementation 3
Core Function of this Subsystem 4
Supervisory Logic Controller (Stateflow) 5
Power Management and Operating States (Stateflow) 6
Mission Termination and Critical Override Logic 7
Navigation and Maneuver Mapping (StateToKinematics) 8
Vehicle Kinematics and Spatial Tracking 9
3D Unreal Engine Visualization and Array Formatting 10
3D Matrix Formatting (pack_3d) 11
Control Script 12
Results and your Conclusion 13

Project Overview & Objective

The Guardian Mission project focuses on developing a comprehensive, system-level digital twin of an autonomous rover. The primary objective of this project is to successfully integrate vehicle kinematics, battery power dynamics, and supervisory state machine logic into a single, unified simulation environment. By bridging mathematical subsystem models with an Unreal Engine 3D visualization, this project validates the rover's behavioral responses, spatial tracking, and power management across continuous mission states.

System Architecture & Methodology

Developed using MATLAB and Simulink (R2025b), the master simulation framework encompasses three core engineering domains:

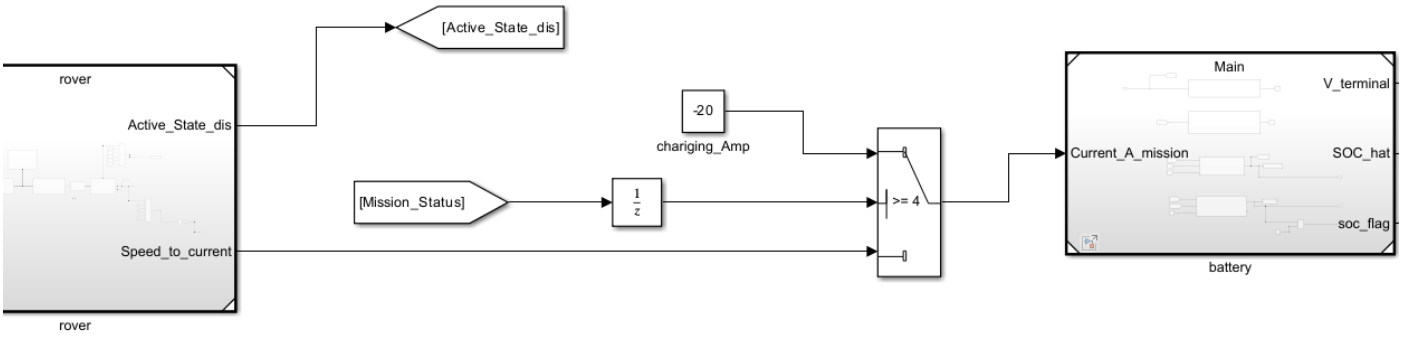
- **Kinematics & Spatial Tracking:** A discrete-time integration subsystem that translates linear velocity (vx) and angular velocity (omega_cmd) commands into real-time X, Y, and Yaw positional coordinates.
- **Supervisory Logic & Power Management:** A Stateflow-driven master controller that governs mission states, evaluates sensor feedback with debounce filtering, and manages the battery equivalent circuit model—including initiating a -20A recovery current upon mission completion.
- **3D Virtual Environment:** A highly constrained physics and visualization engine (Simulation 3D Scene) that requires strict fixed-step solver synchronization, precise multi-dimensional array formatting, and coordinate mapping to render the physical movement of the rover chassis and wheels.

Scope of Implementation

This report details the troubleshooting, synchronization, and final integration phases required to achieve a stable simulation. Key implementations include resolving algebraic loops via unit delays, aligning variable-step and fixed-step solvers across subsystems, and formatting data arrays to bridge the standard Simulink workspace with the 3D visualization engine.

Core Function of this Subsystem

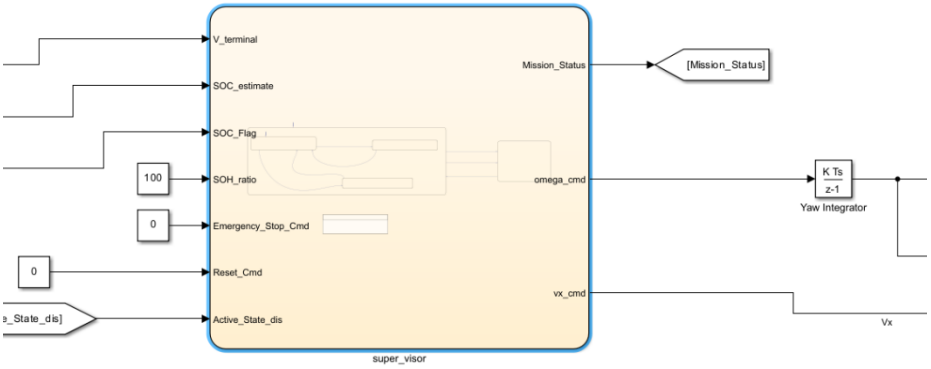
This section of the model acts as the main electrical router between the Rover's driving logic and the Battery's physical response. It dictates whether the battery is being drained by the motors or being actively recharged.



Key Components to Describe:

- **The Switch Block (Current Routing):** A Switch block is utilized to toggle the battery's current load based on the rover's current state.
    - **Normal Driving (Status < 4):** While the rover is actively executing its mission, the switch allows the dynamic Speed_to_current signal (calculated by the motor kinematics) to flow into the battery as a discharge load.
    - **Mission Complete / Charging (Status $\geq$ 4):** Once the Mission_Status reaches 4 or higher (indicating a stop, rest, or base-return state), the switch flips to the top port. It injects a constant **-20 A** (charging_Amp) into the battery model. The negative sign mathematically represents current flowing *into* the battery, simulating a recharge or recovery phase.
- **The Unit Delay Block (1/z) - Simulation Stability:** A discrete Unit Delay block is placed on the Mission_Status control wire. This is a critical simulation mechanism used to break an **Algebraic Loop**. Because the rover's logic relies on battery voltage, and the battery voltage relies on the rover's current draw, instantaneous updates would cause the Simulink solver to freeze or crash. The delay holds the state signal back by exactly one time step, allowing the solver to compute the math sequentially without getting stuck in an infinite loop.
- **Active State Routing:** The Active_State_dis output from the rover is routed into a Goto tag [Active_State_dis]. This cleanly broadcasts the rover's current operational state to the rest of the model for diagnostic monitoring and final post-processing plots without creating messy, overlapping wires across the canvas.

Supervisory Logic Controller (Stateflow)



The super_visor block is a Stateflow chart that serves as the central decision-making "brain" of the Guardian Rover. It continuously evaluates the battery's physical condition and the rover's driving state to dictate the overall mission progression. Based on these evaluations, it generates the specific velocity commands required to physically drive the rover while ensuring safe power management.

Inputs (System Monitoring & Commands) The block receives seven inputs from across the system to inform its logic:

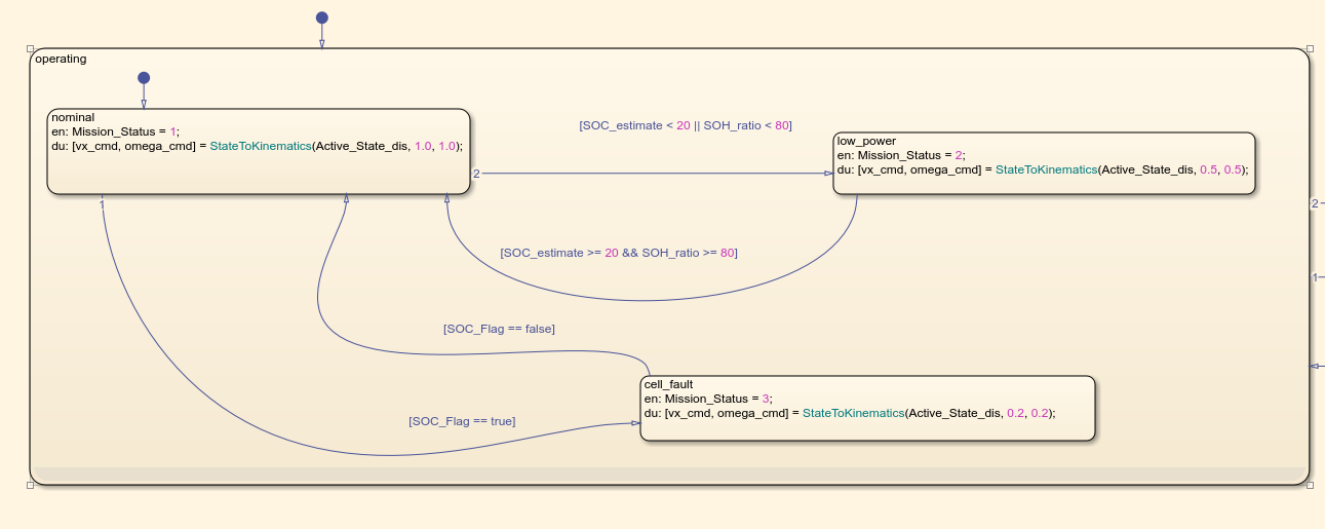
- **V_terminal**: The real-time terminal voltage fed back from the battery equivalent circuit model.
- **SOC_estimate**: The current State of Charge (remaining battery percentage) from the battery's estimator subsystem.
- **SOC_Flag**: A discrete warning signal triggered when the battery drops below a critical threshold, prompting the rover to alter its mission or return to base.
- **SOH_ratio**: The battery's State of Health. For this simulation baseline, it is fed a constant value of 100 to assume a fully healthy, non-degraded battery.
- **Emergency_Stop_Cmd**: An external override to immediately halt the rover. It is held at a constant 0 (disabled) for standard mission execution.
- **Reset_Cmd**: A command to restart the mission logic, held at 0.
- **Active_State_dis**: A feedback loop from the Rover's local navigation logic, telling the supervisor exactly what maneuver the rover is currently performing (e.g., obstacle avoidance, straight driving).

Outputs (Control Signals & Mission Management) The controller outputs three primary signals to drive the rest of the simulation:

- **Mission_Status**: An integer dictating the current phase of the operation. This signal is broadcast via a Goto tag and is critically used to flip the load switch (changing the battery from a discharge state to a -20A charging state when the mission completes).
- **omega_cmd**: The commanded angular velocity (steering rate). This is routed directly into the Yaw Integrator to calculate the rover's heading in the 3D space.

- **vx_cmd**: The commanded linear velocity (forward speed). This is sent to the kinematic equations to calculate the rover's physical X and Y movement over time.

Power Management and Operating States (Stateflow)



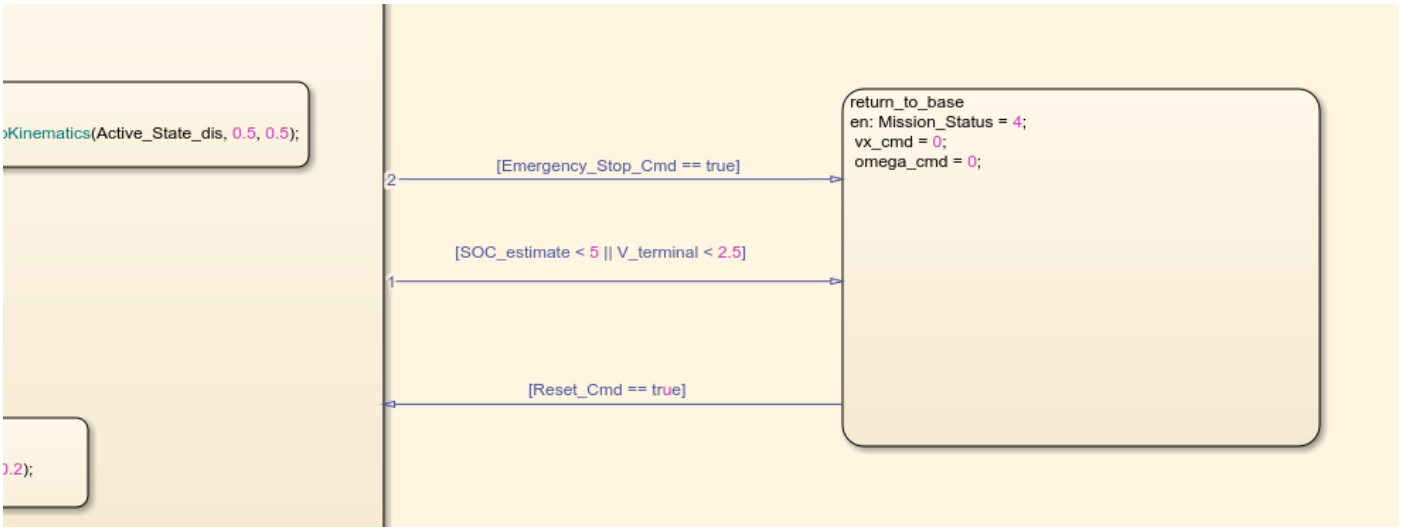
This Stateflow chart defines the internal **operating** logic of the rover. It acts as a dynamic safety and power management system, continuously scaling the rover's physical speed based on the real-time health and charge level of the battery

Operating Modes (States & Transitions) The supervisor dictates the rover's behavior across three distinct operational states:

- **Nominal State (Mission_Status = 1)**
    - **Trigger:** This is the default starting state. The battery is healthy and adequately charged.
    - **Action:** The rover operates at peak performance. The StateToKinematics function applies a 1.0 multiplier (100% speed) to both the linear and angular velocity commands (vx_cmd, omega_cmd).
- **Low Power State (Mission_Status = 2)**
    - **Trigger:** The system transitions to this state if the State of Charge drops below 20% (SOC_estimate < 20) OR if the State of Health degrades below 80% (SOH_ratio < 80).
    - **Action:** To conserve remaining energy, the rover enters a power-saving mode. The velocity multipliers are throttled to 0.5 (50% speed). If the battery is recharged and recovers above these thresholds, the logic allows a transition back to Nominal.
- **Cell Fault State (Mission_Status = 3)**
    - **Trigger:** The system enters this critical state if the battery estimator triggers a fault warning (SOC_Flag == true).
    - **Action:** The rover enters a "limp-home" safety mode. Speeds are severely restricted to a 0.2 multiplier (20% speed) to prevent drawing excessive current from a potentially failing battery.

Action Execution (StateToKinematics) During every time step within these states (du:), the chart calls the custom StateToKinematics function.

Mission Termination and Critical Override Logic



Relationship to the Previous State (operating) The previous section detailed the operating superstate, which handles all active driving phases (Statuses 1, 2, and 3) by scaling speeds based on moderate battery drain. This new image illustrates the **global exit transitions** *out* of that active driving state. It acts as the ultimate safety override: if conditions become too dangerous, the system aborts the entire operating superstate and forces the rover into a terminal return_to_base state.

Mission Abort Triggers (Transitions OUT of Operating) The Stateflow chart continuously monitors for two critical failure conditions that will instantly abort the mission:

- **Critical Battery Depletion:** If the State of Charge drops below 5% (SOC_estimate < 5) OR the battery terminal voltage drops below 2.5V (V_terminal < 2.5), the system forces an abort to prevent permanent hardware damage to the lithium cells.
- **Manual/System Override:** If the Emergency_Stop_Cmd boolean becomes true, the logic bypasses all standard operations and immediately terminates the driving phase.

The return_to_base State (Status = 4) When either critical trigger is hit, the rover enters the return_to_base block, executing three immediate actions:

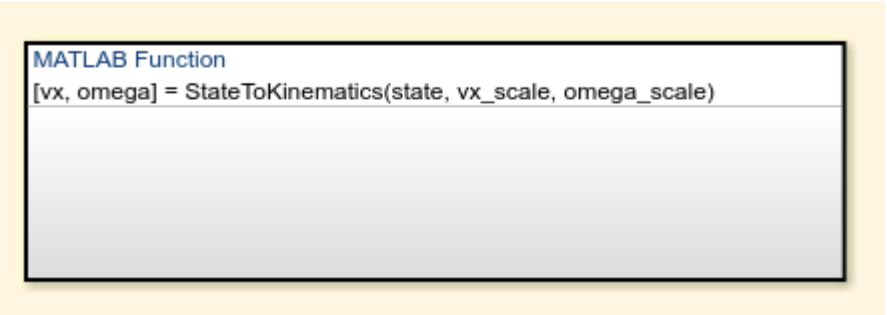
1. **vx_cmd = 0; and omega_cmd = 0;**: Linear and angular velocity commands are forced to zero, bringing the rover's physical motors to a complete and immediate stop.
2. **Mission_Status = 4;**: The state machine broadcasts a status of 4 to the rest of the Simulink model.
3. **System-Wide Tie-In:** As discussed in the Power Management section of this report, outputting Mission_Status = 4 is the exact trigger that flips the external Simulink switch. This disconnects the motor load and injects the **-20A** charging/recovery current into the battery model.

System Recovery (Transition INTO Operating) The rover remains locked in this halted, charging state until it receives a manual Reset_Cmd == true, at which point the logic is allowed to transition back into the operating superstate to resume driving.

Navigation and Maneuver Mapping (StateToKinematics)

```
function [vx, omega] = StateToKinematics(state, vx_scale, omega_scale)

if state == 1
    vx = 2.0 * vx_scale;
    omega = 0.0;
elseif state == 2
    vx = 0.0;
    omega = 1.0 * omega_scale;
elseif state == 3
    vx = 0.0;
    omega = -1.0 * omega_scale;
else
    % Covers 0 (IDLE) and 4 (STOP)
    vx = 0.0;
    omega = 0.0;
end
```



The StateToKinematics MATLAB Function block acts as the exact mathematical translator between the rover's high-level logical decisions and its physical motors. It takes the current navigational goal (the state—such as moving forward or turning) and applies the power-saving multipliers (vx_scale, omega_scale) assigned by the supervisory controller. It then outputs the final linear (vx) and angular (omega) velocities to the physical kinematics equations.

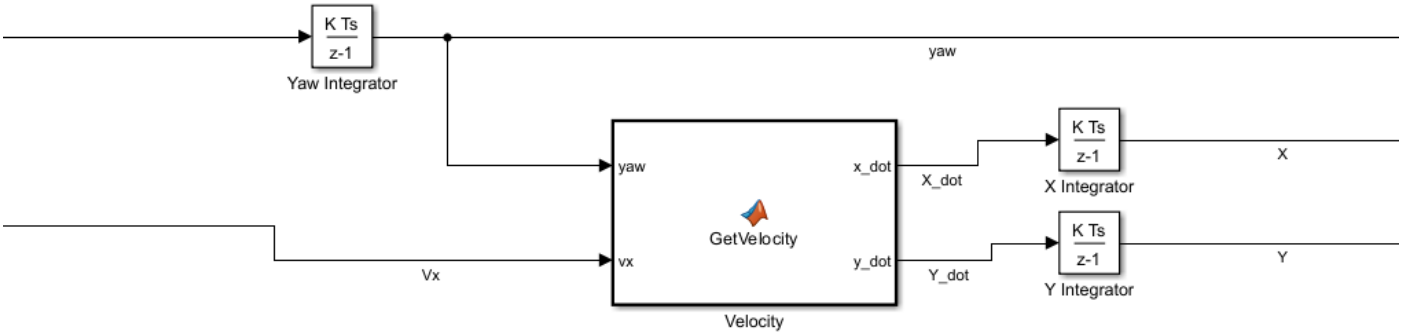
Maneuver Mapping (Logic to Velocity) The function uses an if-elseif structure to define the baseline velocities for four distinct local navigation states:

- **State 1 (Straight Forward):**
    - The rover aims to drive straight ahead.
    - **Action:** It sets a baseline forward speed of 2.0 and multiplies it by the current vx_scale. The turning speed (omega) is locked to 0.0 to maintain a straight path.
- **State 2 (Left Turn):**
    - The rover has detected an obstacle and needs to navigate left.
    - **Action:** It stops forward movement (vx = 0.0) and initiates a positive (counter-clockwise) rotation at a baseline speed of 1.0, multiplied by the omega_scale.
- **State 3 (Right Turn):**
    - The rover needs to navigate right.

- **Action:** It stops forward movement (vx = 0.0) and initiates a negative (clockwise) rotation at a baseline speed of -1.0, multiplied by the omega_scale.

- **Default State (Idle / Stop):**
  - This covers State 0 (Initial Idle) and State 4 (Mission Stop / Return to Base).
  - **Action:** Both vx and omega are forced to 0.0, bringing the rover to a complete physical standstill regardless of what the supervisory multipliers are doing.

Vehicle Kinematics and Spatial Tracking



This section of the model acts as the kinematic physics engine for the rover. It takes the raw, local driving commands (forward speed and turning rate) generated by the Stateflow controller and mathematically translates them into global spatial coordinates (X, Y, and Yaw). These coordinates are required to map the rover's physical movement inside the 3D Unreal Engine environment.

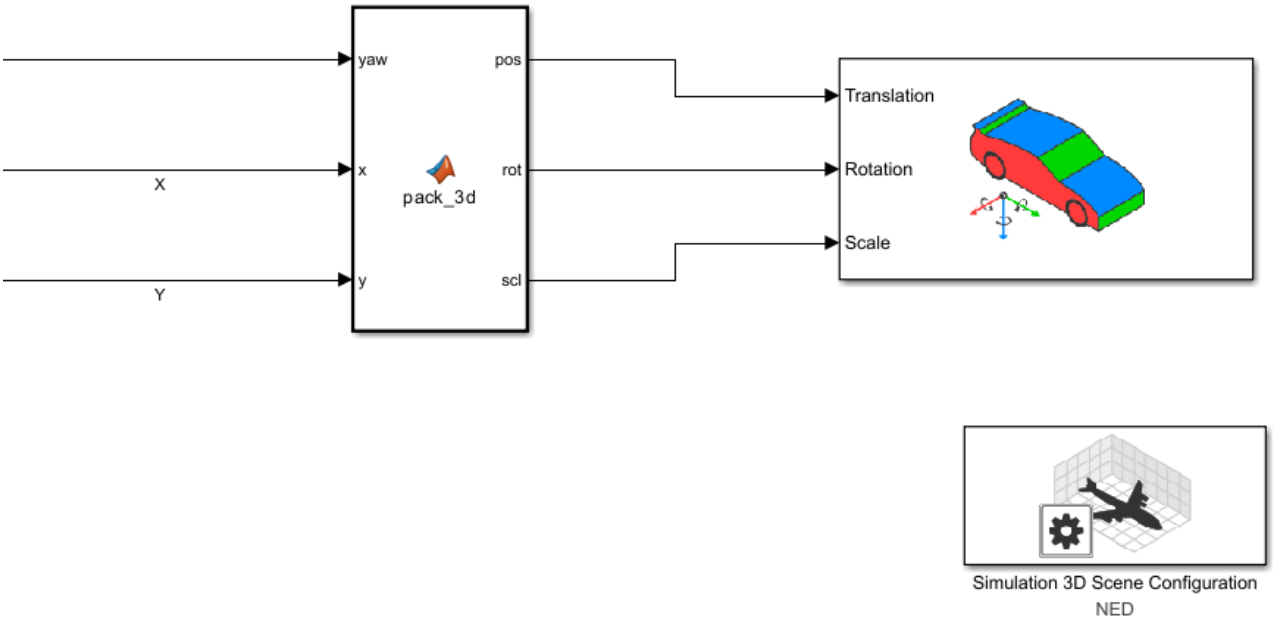
Mathematical Workflow & Component Breakdown The subsystem calculates the vehicle's position through three sequential steps:

- **1. Heading Calculation (Yaw Integrator):** The commanded angular velocity (omega_cmd, coming from the left) is fed into a Discrete-Time Integrator (K Ts / z-1) named **Yaw Integrator**. Integrating the rover's turning rate over time calculates the vehicle's absolute heading angle, labeled as the **yaw** signal.

- **2. Velocity Decomposition (GetVelocity):** The **GetVelocity** MATLAB Function block serves as a trigonometric resolver. It receives the rover's linear forward speed (**Vx**) and its current heading (**yaw**). Inside this block, the local forward speed is projected onto the global axes using standard unicycle kinematics:
  - $x_dot = Vx \cdot \cos(yaw)$
  - $y_dot = Vx \cdot \sin(yaw)$ This outputs the real-time velocity of the rover along the global X and Y axes (**x_dot** and **y_dot**).

- **3. Global Position Tracking (X and Y Integrators):** The decomposed directional velocities are fed into two final Discrete-Time Integrators (**X Integrator** and **Y Integrator**). By integrating the velocity over time, these blocks continuously calculate the rover's exact **X** and **Y** positional coordinates on the map.

3D Unreal Engine Visualization and Array Formatting



This section serves as the bridge between the mathematical simulation and the virtual 3D environment. It formats the 2D positional data into the strict multi-dimensional matrices required by the Unreal Engine, and initializes the physical rendering of the vehicle.

Component Breakdown

- **Data Formatting (pack_3d Function):** The 3D Vehicle block in modern Simulink versions requires a 5×3 matrix for its inputs to control the vehicle chassis (Row 1) and its four independent wheels (Rows 2-5). The pack_3d MATLAB function takes the scalar X, Y, and yaw values from the kinematics and packs them into these required matrices (pos, rot, scl). Crucially, it locks the translation and rotation offsets for the wheels to [0, 0, 0] relative to the chassis to prevent physical detachment anomalies during rendering.
- **Vehicle Rendering (3D Vehicle Block):** This block represents the physical rover (rendered as the red muscle car). It receives the formatted Translation, Rotation, and Scale matrices and applies them directly to the 3D mesh frame by frame, allowing the vehicle to drive and turn smoothly across the virtual landscape.
- **Environment Initialization (Simulation 3D Scene Configuration):** This block establishes the connection to the Unreal Engine. It configures the physics engine, loads the sky and ground textures, and enforces the North-East-Down (NED) coordinate system. It also dictates the strict fixed-step solver requirement that forces the rest of the Simulink model to synchronize to a single clock rate.

3D Matrix Formatting (pack_3d)

```
function [pos, rot, scl] = pack_3d(yaw,x, y)
    % Row 1 is the car body (it gets the real coordinates)
    % Rows 2-5 are the wheels (0 offset keeps them attached to the car)
    pos = [x, y, 0;
           0, 0, 0;
           0, 0, 0;
           0, 0, 0;
           0, 0, 0];

    rot = [0, 0, yaw;
           0, 0, 0;
           0, 0, 0;
           0, 0, 0;
           0, 0, 0];

    % Scale remains 1 for all parts so they don't disappear
    scl = [1, 1, 1;
           1, 1, 1;
           1, 1, 1;
           1, 1, 1;
           1, 1, 1];
end
```

The pack_3d MATLAB Function is a critical data-structuring script. The Unreal Engine 3D Vehicle block requires precise 5×3 matrices to individually control the vehicle's chassis (Part 1) and its four separate wheels (Parts 2-5). This function converts the scalar x, y, and yaw variables from the kinematics model into these required multi-dimensional arrays (pos, rot, scl).

Array Construction & Wheel Attachment Logic

- **Position (pos):** The first row receives the live [x, y, 0] coordinates to translate the main car body across the map. Rows 2 through 5 represent the four wheels. By forcing the wheel position offsets to [0, 0, 0], the script physically locks the wheels to the moving chassis, resolving a physics bug where the engine would add the coordinates twice and cause the wheels to detach and drive away.
- **Rotation (rot):** The live yaw angle is applied to the Z-axis of the first row ([0, 0, yaw]), allowing the vehicle chassis to steer and rotate. The wheel rotations (rows 2-5) are locked to [0, 0, 0]. Because the simplified kinematics model calculates a whole-body heading rather than individual tire pitch/spinning, this keeps the wheels visually aligned with the chassis as it glides over the terrain.
- **Scale (scl):** All five rows are set to a constant [1, 1, 1]. This ensures the 3D meshes for the chassis and all four wheels render at exactly 100% of their normal visual size and do not collapse or disappear from the simulation.

Control Script

```
%% Guardian Mission - Master Initialization and Control Script
clear; clc; close all;
disp('Initializing Workspace...');

% 1. Add the subfolders to MATLAB's search path
addpath(fullfile(pwd, 'Rover'));
addpath(fullfile(pwd, 'Battery'));
N_confirm = 5;

% 2. Load Variables
% (Because we added the paths above, we can just call the script names directly)
Base
run_MAIN

% 3. Run the Top-Level Harness
disp('Starting 3D Simulation...');
simOut = sim('finals.slx');
disp('Mission Complete!');
```

This top-level MATLAB script acts as the executive controller for the entire Guardian Mission simulation. It prepares the clean workspace, links the isolated subsystem directories, loads all required physical and electrical parameters, and commands Simulink to execute the master 3D model.

Script Execution Flow

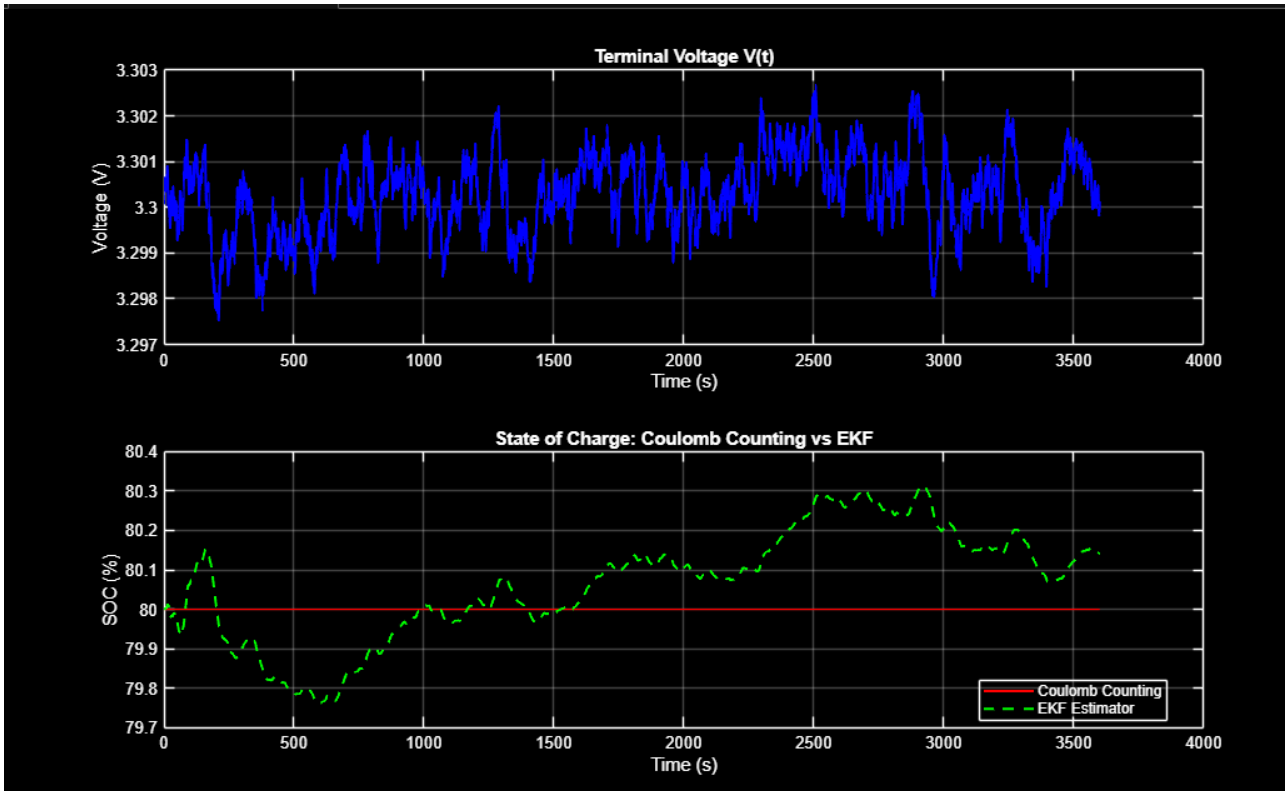
- **Workspace & Path Management:** The script begins by clearing legacy data (clear; clc; close all;) and using the addpath command to dynamically link the Rover and Battery subfolders. This is a critical step that prevents "file not found" errors when the master model attempts to load the referenced subsystems.
- **Logic Initialization:** It explicitly defines N_confirm = 5. This establishes the debounce threshold for the sensor processing subsystem, ensuring the rover filters out temporary signal noise before reacting to an obstacle.
- **Variable Loading:** The script executes the Base and run_MAIN sub-scripts. These actions populate the base MATLAB workspace with the exact kinematic dimensions and battery equivalent circuit variables required by the Simulink blocks.
- **Simulation Execution:** Finally, it triggers the top-level model using simOut = sim('finals.slx'). This launches the synchronized fixed-step solver, starts the Unreal Engine 3D visualization, and captures the resulting trajectory and power data into the simOut object for post-processing.

Results and your Conclusion

The Guardian Mission digital twin was successfully implemented and validated. The integration of the physical battery equivalent circuit model with the supervisory Stateflow controller allowed the rover to dynamically adjust its navigation parameters based on real-time power constraints. By synchronizing fixed-step solvers and formatting kinematic arrays for the Unreal Engine environment, the simulation achieved stable, continuous 3D visualization without algebraic loops or matrix dimension errors. The final system effectively demonstrates the principles of Model-Based Design for autonomous vehicle power and spatial management.

```
Simulation completed successfully! Launching post-processing...
Plotting results and calculating SOH...

=====================================
Bonus 3 Results (TLS Identification):
Estimated R0 (TLS) = 0.000700 Ohms
State of Health (SOH) = 100.00 %
=====================================
Parameters and Bus Objects loaded successfully.
Starting 3D Simulation...
```




Simulation 3D Viewer (64-bit Development PCD3D_SM5)